

CS471 – Web Technologies

Laboratory – Lab 1

The Internet Protocols

Student Name	Shatha Almutairi
Student ID	432205190
Course	CS471 – Web Technologies (Laboratory)
Lab Title	Lab 1 – The Internet Protocols
Institution	Qassim University – College of Computer

Lab Objectives

This lab session covers the usage of the Wireshark application to monitor and capture the outgoing and incoming packets from a network connection (Wi-Fi, Ethernet, etc.). Specifically, students should be able to analyze HTTP, HTTPS, TCP/IP, and UDP protocols using Wireshark, a network protocol analyzer, and draw conclusions.

Pre-lab Preparation

Question / Step:

1. Review the basics and the structure of HTTP, TCP/IP, and UDP protocols.
2. Install Wireshark and ensure it is running on your computer.
3. Create an online, publicly accessible Git repository to host and upload your work in the labs (GitHub or GitLab is recommended).

Answer / Result:

The structure of the HTTP, TCP/IP, and UDP protocols was reviewed before the session.

Wireshark was installed successfully and was running on the computer during the lab.

A publicly accessible online Git repository was created, and the lab screenshots and the final report were uploaded to it.

Part 1: Capturing HTTP Traffic

Task 1: Start Wireshark and capture packets

Question / Step:

Step 1: Open Wireshark.

Step 2: Select the network interface connected to the internet (e.g., Ethernet or Wi-Fi).

Step 3: Click the "Start Capturing Packets" button (the shark fin icon).

Step 4: Open your favorite web browser and navigate to (<http://neverssl.com/>) website.

Step 5: After the website has fully loaded, stop capturing packets by clicking the red stop button in Wireshark.

Answer / Result:

Wireshark was opened and the active Wi-Fi interface was selected as the capture interface.

Packet capturing was started using the shark fin icon, then the browser was used to navigate to <http://neverssl.com/>.

After the page had fully loaded, the capture was stopped with the red stop button. The captured traffic contained the HTTP packets exchanged with the website.

Task 2: Filter HTTP packets and analyze them

Question / Step:

Step 1: In the filter bar, type http and press Enter. This filters out only the HTTP packets from the capture.

Answer / Result:

The display filter http was applied, and only the HTTP packets remained visible in the packet list, as shown below.

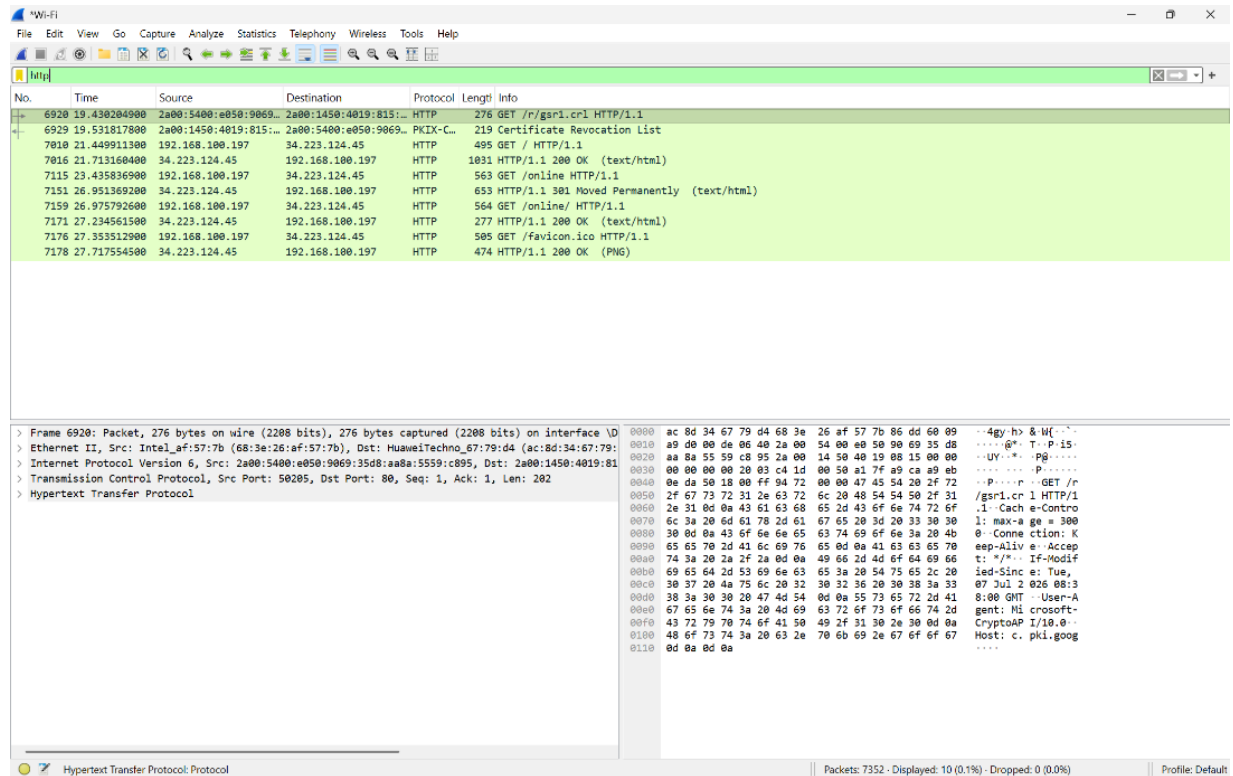


Figure 1: Applying the "http" display filter in Wireshark.

Question / Step:

Step 2: Select any HTTP packet to view its details.

Answer / Result:

An HTTP packet was selected, and its details were expanded in the packet details pane (Frame, Ethernet II, IP, TCP, and Hypertext Transfer Protocol layers).

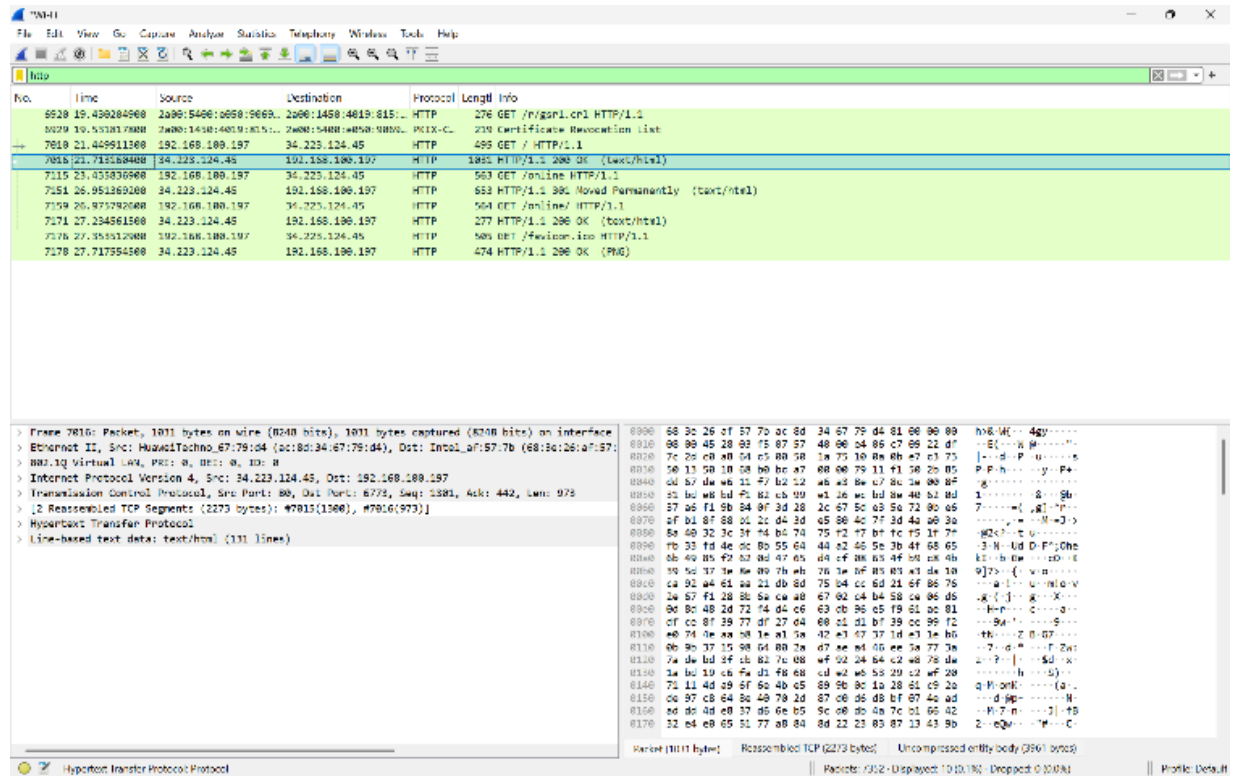


Figure 2: Details of a selected HTTP packet.

Question / Step:

Step 3: Observe the HTTP request and response messages. Note the method (GET, POST), URL, response codes (200 OK, 404 Not Found), etc.

Answer / Result:

- HTTP Method: GET
- URL/Path: /
- Response Code: 200 OK
- Protocol: HTTP/1.1

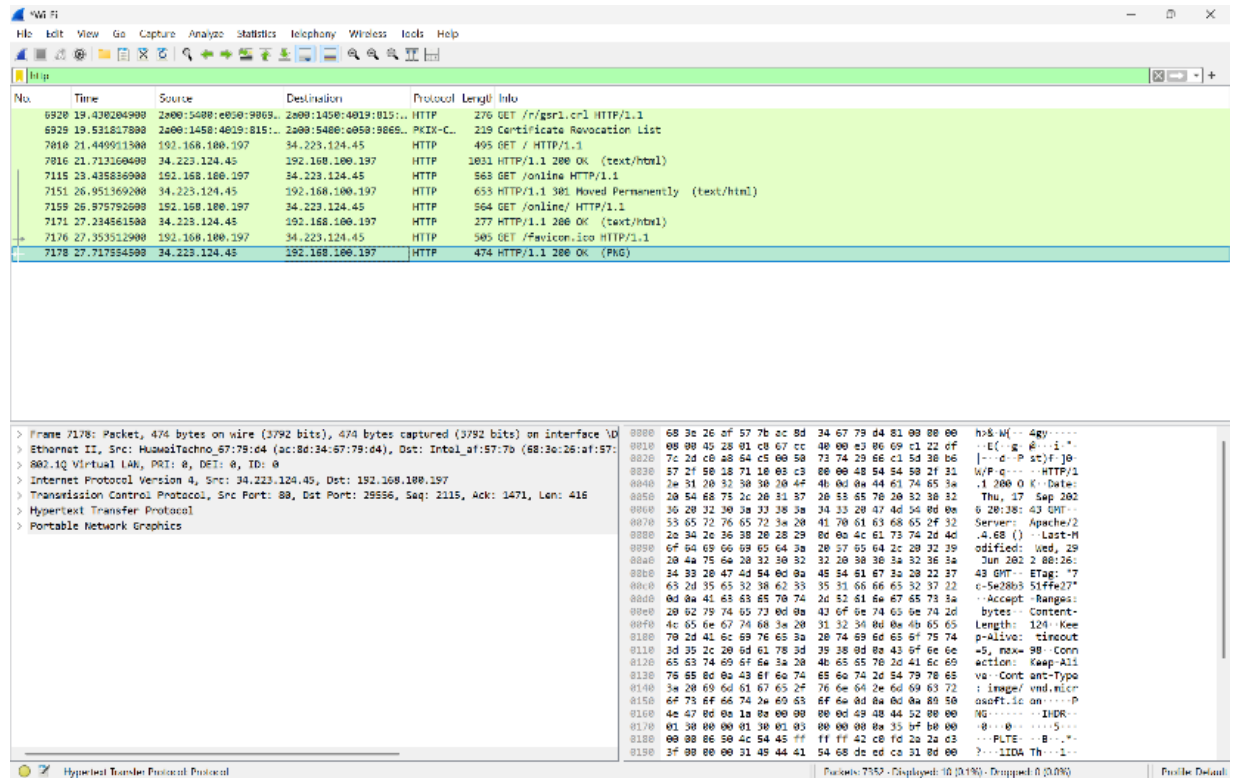


Figure 3: HTTP request and response messages showing GET / and 200 OK.

Part 2: Analyzing TCP/IP Traffic

Task 1: Filter TCP packets

Question / Step:

Step 1: Clear the previous filter and type TCP to focus on TCP packets.

Answer / Result:

The previous filter was cleared and the tcp filter was applied, so only TCP packets were displayed.

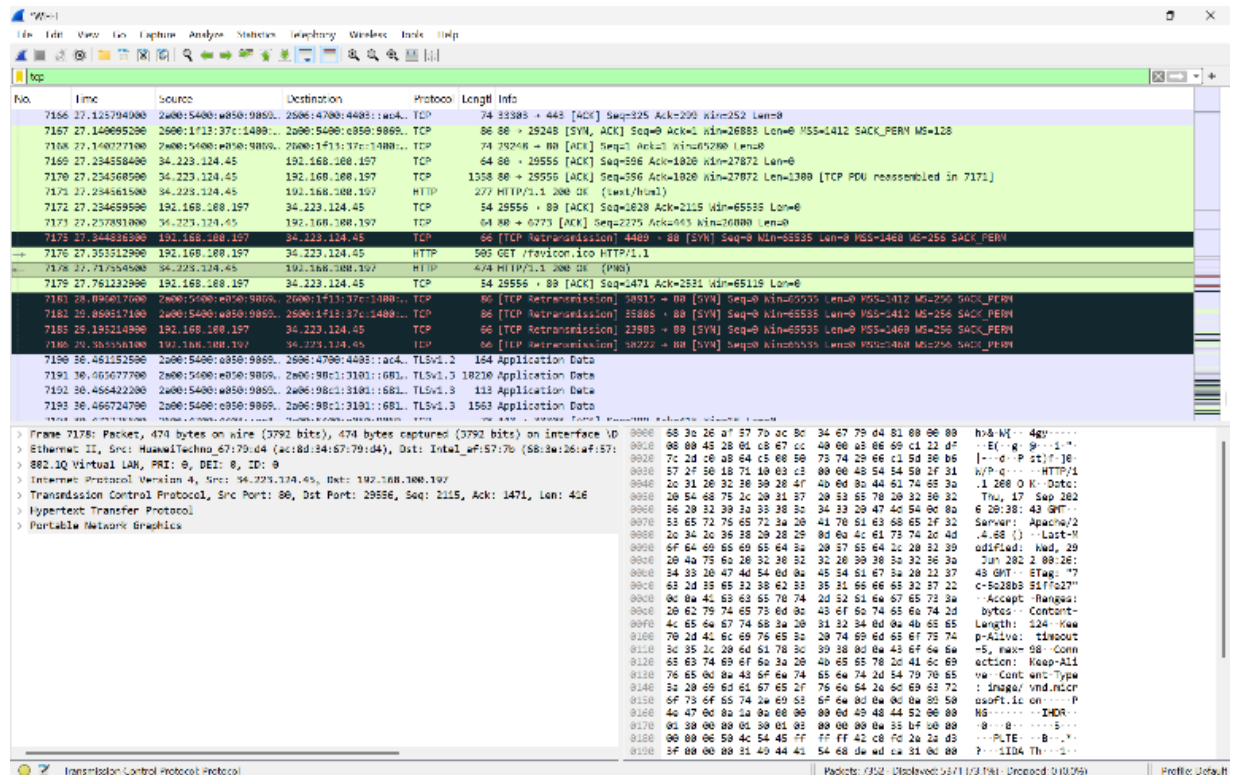


Figure 4: Applying the "tcp" display filter.

Question / Step:

Step 1: Find and select packets related to the TCP three-way handshake:

- SYN: Initiates a connection.
- SYN-ACK: Acknowledges and responds to the SYN.
- ACK: Acknowledges the SYN-ACK and establishes the connection.

Answer / Result:

The three packets of the TCP three-way handshake were located in the capture using the tcp.flags filter.

No.	Time	Source	Destination	Protocol	Length	Info
7072	22.178408280	192.168.100.197	34.223.124.45	TCP	66	29556 → 80 [SYN] Seq=0 Win=65535 Len=0 MSS=1460 WS=256 SACK_PERM
7106	23.191235500	192.168.100.197	34.223.124.45	TCP	66	[TCP Retransmission] 29556 → 80 [SYN] Seq=0 Win=65535 Len=0 MSS=1460 WS=256 SACK_PERM
7113	23.435027000	34.223.124.45	192.168.100.197	TCP	62	80 → 29556 [SYN, ACK] Seq=0 Ack=1 Win=20883 Len=0 MSS=1412
7114	23.435178400	192.168.100.197	34.223.124.45	TCP	54	29556 → 80 [ACK] Seq=1 Ack=1 Win=65535 Len=0
7115	23.435855600	192.168.100.197	34.223.124.45	HTTP	668	GET /online HTTP/1.1
7148	26.795259200	192.168.100.197	34.223.124.45	TCP	66	[TCP Retransmission] 29556 → 80 [SYN, ACK] Seq=1 Ack=1 Win=65535 Len=0
7149	26.795169300	34.223.124.45	192.168.100.197	TCP	58	80 → 29556 [ACK] Seq=1 Ack=510 Win=20800 Len=0
7151	26.951369200	34.223.124.45	192.168.100.197	HTTP	653	HTTP/1.1 301 Moved Permanently (text/html)
7159	26.975792600	192.168.100.197	34.223.124.45	HTTP	564	GET /online/ HTTP/1.1
7169	27.234558400	34.223.124.45	192.168.100.197	TCP	64	80 → 29556 [ACK] Seq=596 Ack=1820 Win=27872 Len=0
7170	27.234568500	34.223.124.45	192.168.100.197	TCP	1358	80 → 29556 [ACK] Seq=596 Ack=1820 Win=27872 Len=1308 [TCP PDU reassembled in 7171]
7171	27.234561500	34.223.124.45	192.168.100.197	HTTP	277	HTTP/1.1 200 OK (text/html)
7172	27.234659500	192.168.100.197	34.223.124.45	TCP	54	29556 → 80 [ACK] Seq=1820 Ack=2115 Win=65535 Len=0
7176	27.353512000	192.168.100.197	34.223.124.45	HTTP	505	GET /favicon.ico HTTP/1.1
7178	27.717554500	34.223.124.45	192.168.100.197	HTTP	474	HTTP/1.1 200 OK (PNG)
7179	27.761232900	192.168.100.197	34.223.124.45	TCP	54	29556 → 80 [ACK] Seq=1471 Ack=2531 Win=65119 Len=0
7232	32.632434400	34.223.124.45	192.168.100.197	TCP	58	80 → 29556 [FIN, ACK] Seq=2531 Ack=1471 Win=28944 Len=0
7233	32.632519900	192.168.100.197	34.223.124.45	TCP	54	29556 → 80 [ACK] Seq=1471 Ack=2532 Win=65119 Len=0

Figure 7: The TCP three-way handshake: SYN, SYN-ACK, and ACK packets.

Question / Step:

Step 2: Note the sequence and acknowledgment numbers. Screenshot and upload your image to your online git repository.

Answer / Result:

- SYN: Seq = 0, Ack = 0
- SYN-ACK: Seq = 0, Ack = 1
- ACK: Seq = 1, Ack = 1

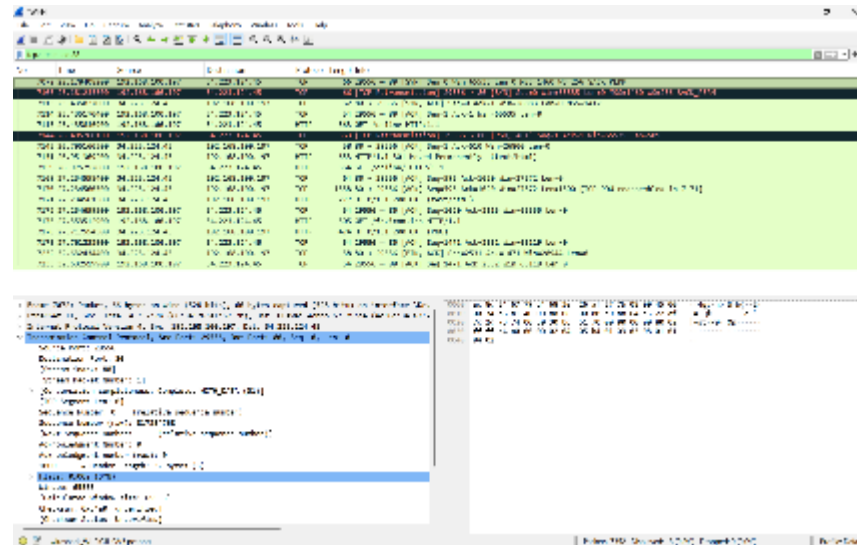


Figure 8: SYN packet – sequence and acknowledgment numbers.

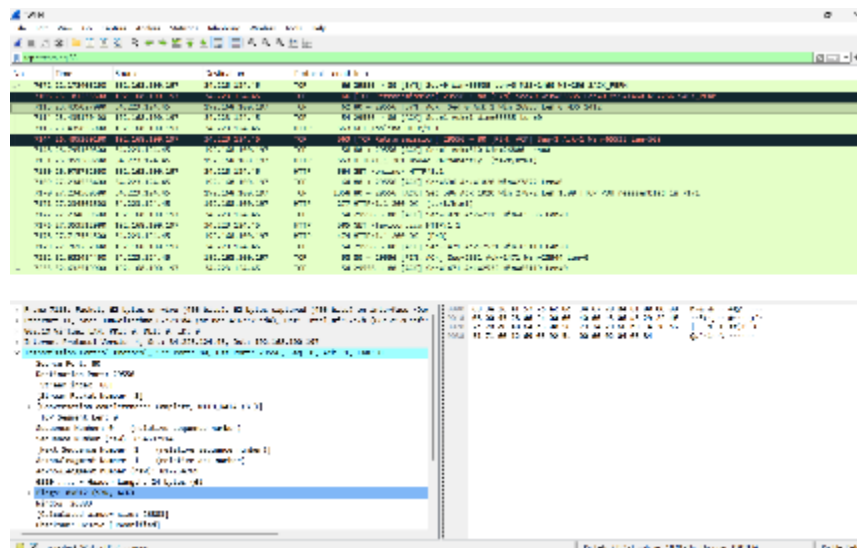


Figure 9: SYN-ACK packet – sequence and acknowledgment numbers.

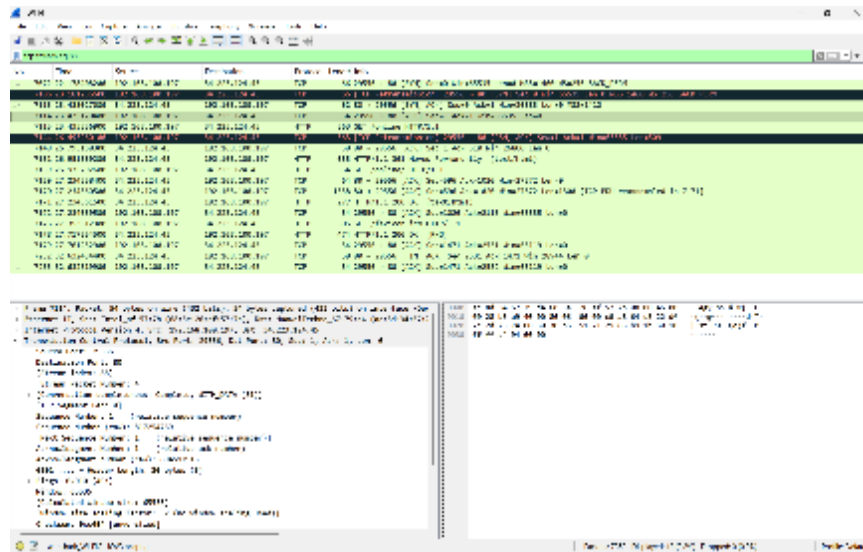


Figure 10: ACK packet – sequence and acknowledgment numbers.

Question / Step:

Step 3: Observe the data packets exchanged between the client and server. Take a screenshot and upload it to your online git repo.

Answer / Result:

After the connection was established, the data transfer phase was observed: the client sent the HTTP GET request and the server replied with the HTTP response, with each segment acknowledged by the receiver.

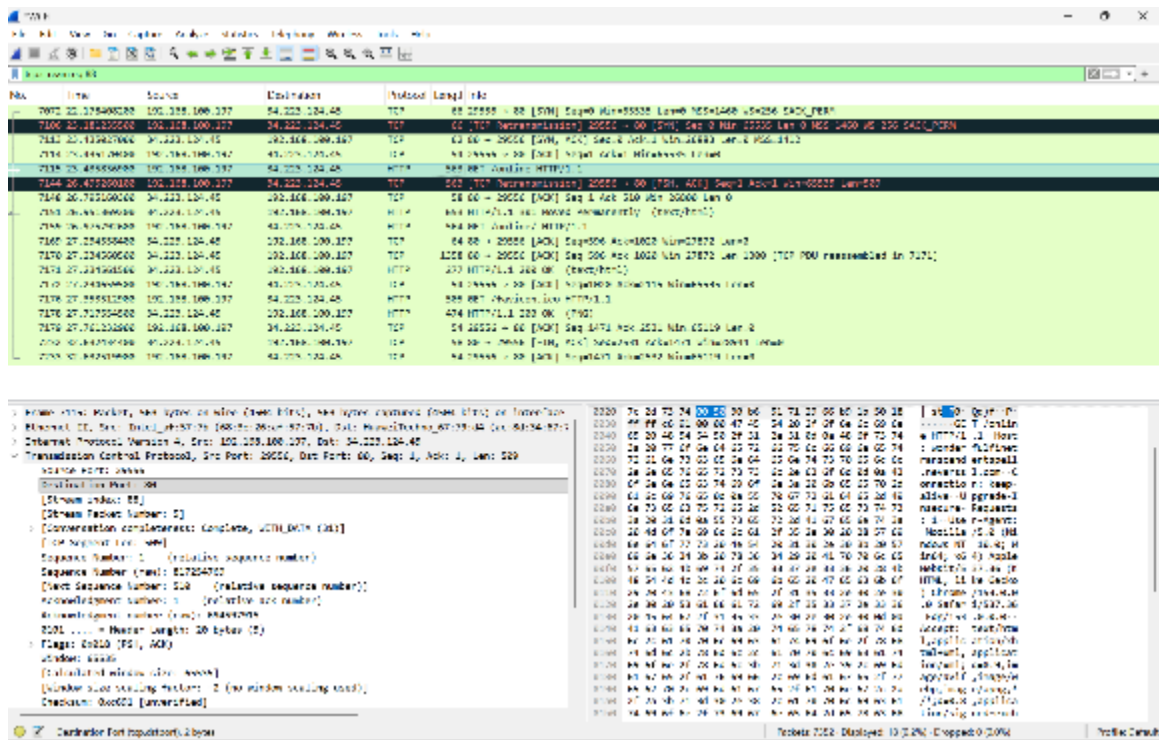


Figure 11: Data transfer: the HTTP GET request and the HTTP response between client and server.

Part 3: Capturing and Analyzing UDP Traffic

Task 1: Generate UDP traffic and capture packets

Question / Step:

Step 1: Open a network application that uses UDP (e.g., streaming video, VoIP software, or custom script).

Answer / Result:

A streaming video application (an online video in the browser) was opened, since video streaming uses UDP for data delivery.



Figure 13: The streaming video application used to generate UDP traffic.

Question / Step:

Step 2: Start the application to generate UDP traffic.

Answer / Result:

The video was played to generate a continuous flow of UDP traffic.

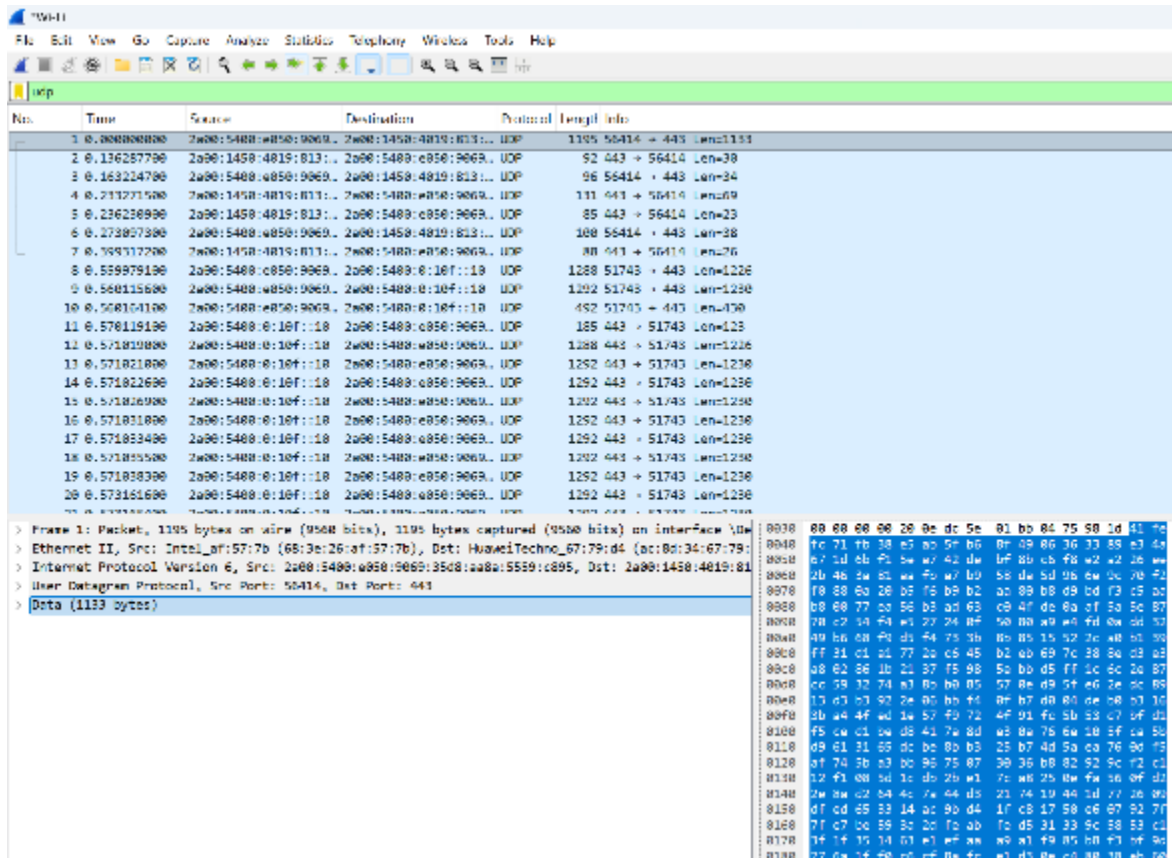


Figure 14: UDP traffic generated by the streaming application.

Question / Step:

Step 3: Start capturing packets in Wireshark while the UDP application is running.

Answer / Result:

Capturing was started in Wireshark while the streaming application was still running, so the UDP packets were recorded.

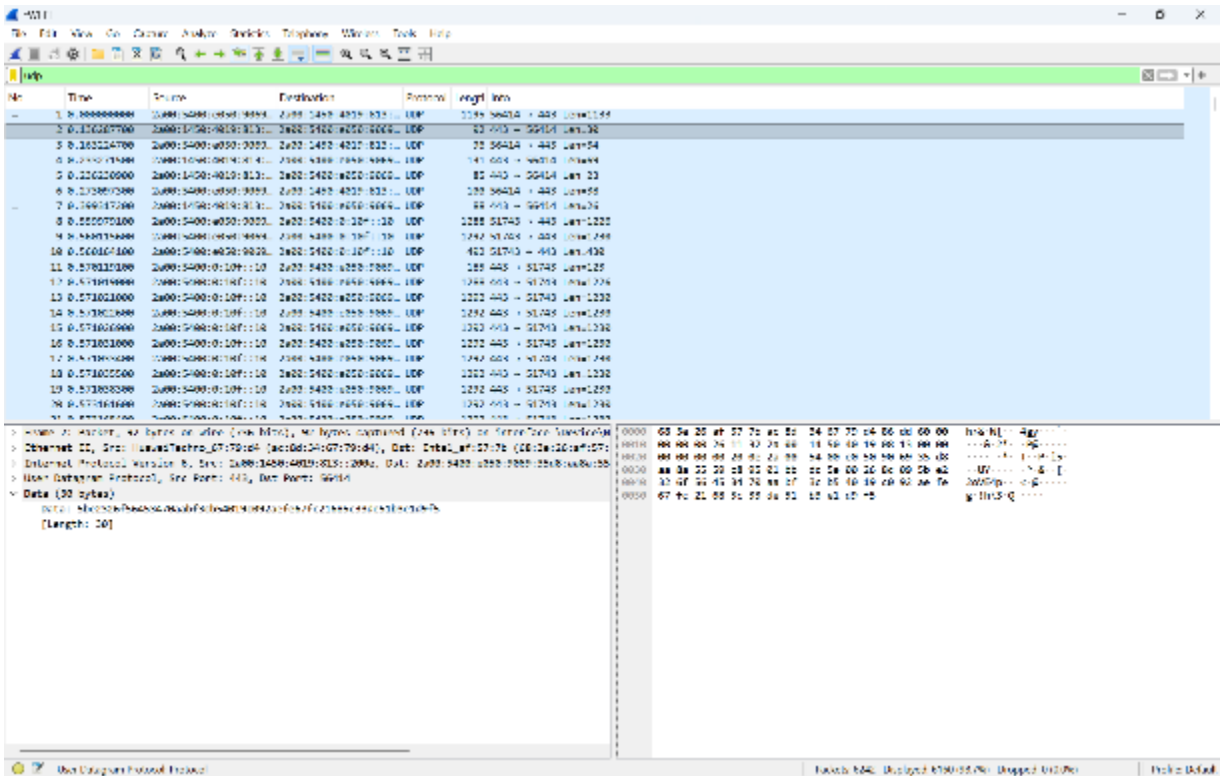


Figure 15: Capturing packets in Wireshark while the UDP application is running.

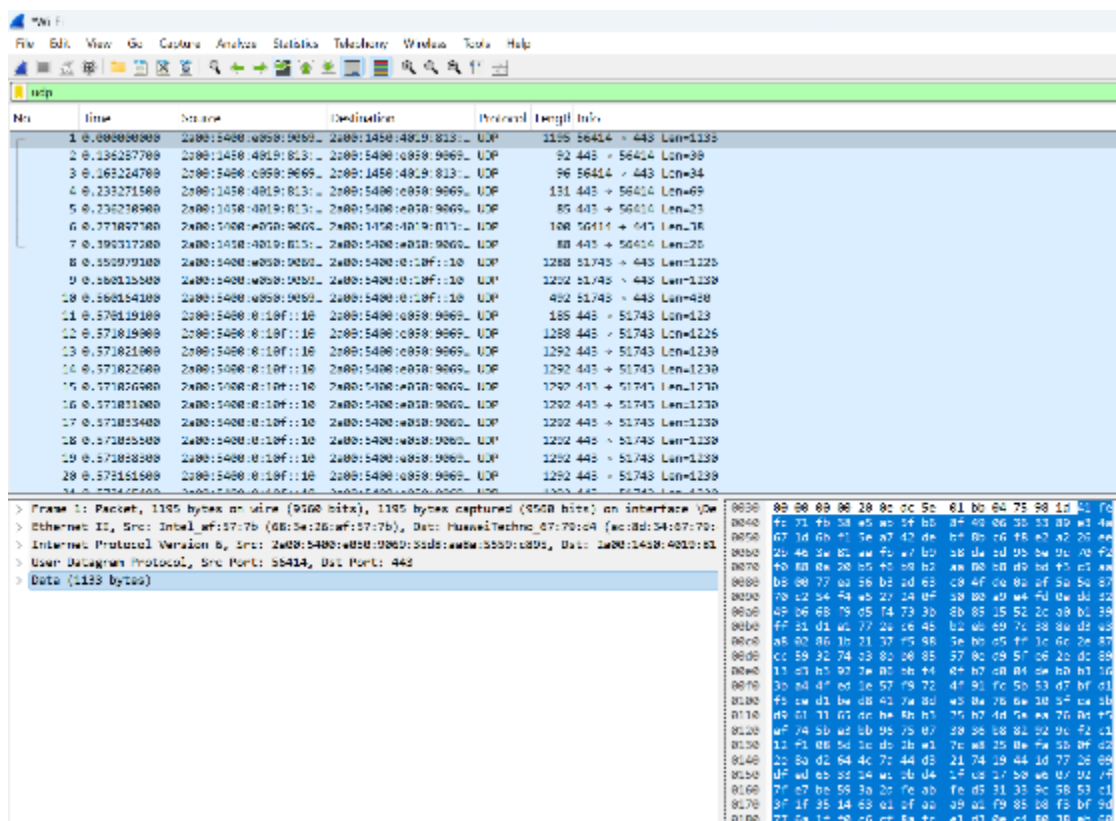


Figure 18: Only UDP packets are displayed after filtering.

Question / Step:

Step 3: Select any UDP packet to view its details.

Answer / Result:

A UDP packet was selected and the User Datagram Protocol layer was expanded in the packet details pane.

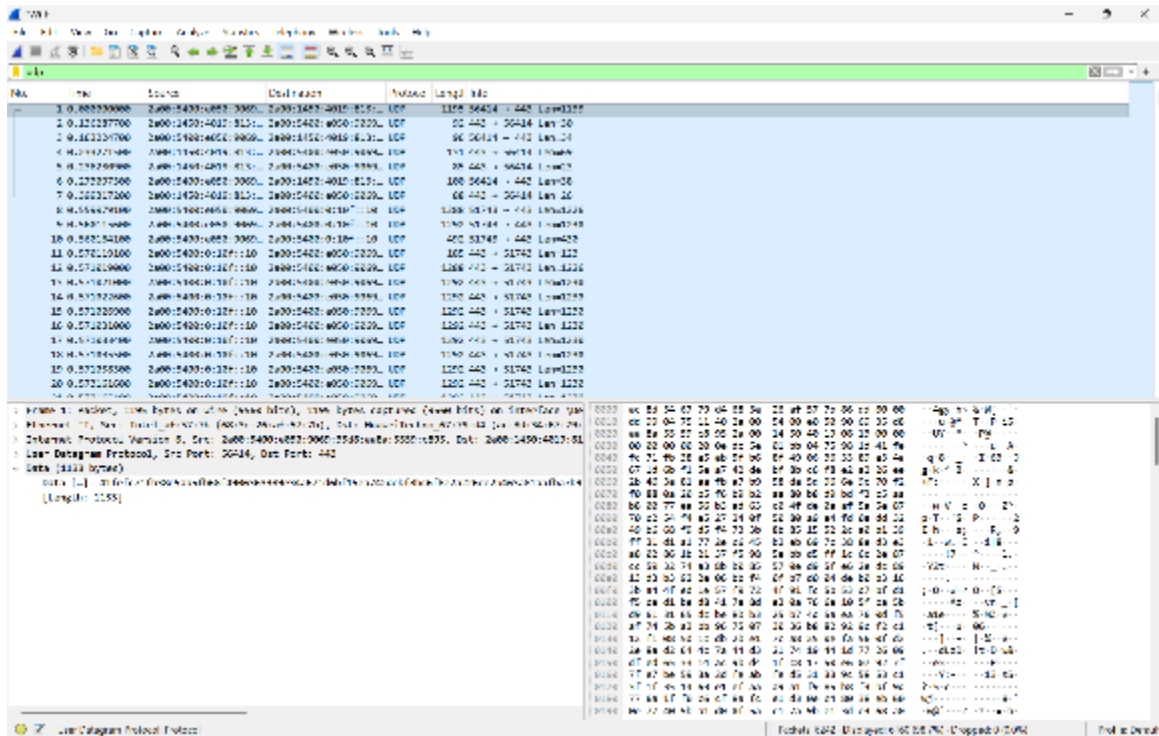


Figure 19: Details of a selected UDP packet.

Question / Step:

Step 4: Observe the source and destination ports, length, and data.

Answer / Result:

- Source Port: 56414
- Destination Port: 443
- Length: 1141 bytes
- Data: 1133 bytes

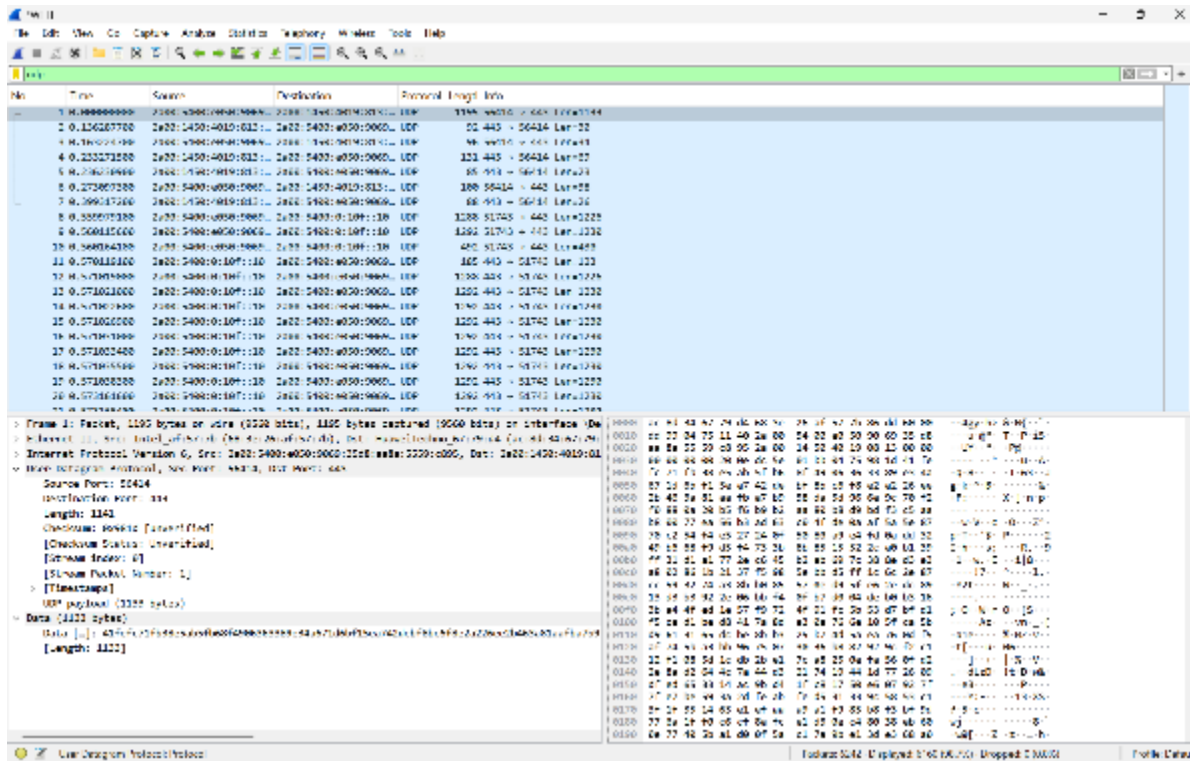


Figure 20: UDP header fields: source port, destination port, length, and data.

Question / Step:

Step 5: Compare the simplicity of UDP headers with TCP headers.

Answer:

UDP header is simpler and smaller than TCP header. UDP has fewer fields and does not use connection establishment, sequence numbers, acknowledgments, or retransmission. TCP header is more complex because it provides reliable and ordered delivery.

Part 4: Comparing TCP and UDP

Comparing TCP and UDP by filling in the following tables. Save your work (e.g., in an MS Word document), and upload it to your online git repo.

Task 1: Fill in the following table and provide reasons

Question / Step:

Fill in the following table and provide reasons for Reliability and Connection Establishment, and for Data Integrity and Ordering.

Answer / Result:

	TCP or UDP	Reasons
Reliability and Connection Establishment	TCP	TCP establishes a connection before sending data and ensures reliable delivery.
Data Integrity and Ordering	TCP	TCP checks data delivery and keeps packets in the correct order.

Task 2: Identify the use cases and performance of TCP and UDP

Question / Step:

Identify the use cases and performance of TCP and UDP.

Answer / Result:

	TCP	UDP
Use cases	Web browsing, Email, File transfer	Streaming, Online gaming, VoIP
Performance	Slower because it checks delivery and ordering.	Faster because it has less overhead and no connection setup.

Conclusion

In this lab, Wireshark was used to capture and analyze real network traffic. HTTP traffic was captured from <http://neverssl.com/> and the request method, URL, response code, and protocol version were identified. The TCP analysis showed the three-way handshake, the data transfer phase, and the connection termination with FIN and ACK packets. Finally, UDP traffic generated by a streaming application was captured and its header fields were examined, which confirmed that the UDP header is much simpler than the TCP header because UDP does not provide reliability, ordering, or connection establishment.